18.4 树形DP

18.4.1 定义

树上进行的、父节点依赖子节点的DP

18.4.2 特点

- 因为是在搜索树上进行的，这类问题我们往往会用记忆化搜索来实现
- 初值非常重要，边界就是递归到树的叶节点

提示：

可以多画画图帮助理清思路

没有上司的舞会（洛谷1352）

【题目描述】

[P1352 没有上司的舞会 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://www.luogu.com.cn/problem/P1352)

【解题思路】

[P1352 没有上司的舞会 - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://www.luogu.com.cn/problem/P1352)

例题 UVA12186

【题目描述】

某公司里有一个老板和 n（$n \leq 10^5$） 个员工组成树状结构，除了老板之外每个员工都有唯一的直属上司。老板的编号为 0 ，员工编号为 $1 \sim n$ 。工人们（即没有直接下属的员工）打算签署一项请愿书递给老板，但是不能跨级递，只能递给直属上司。当一个中级员工（不是工人的员工）的直属下属中不小于 T 的人签字时，他也会签字并且递给他的直属上司。问： 要让公司老板收到请愿书，至少需要多少个工人签字?

【解题思路】

$f(i)$ 返回 i 为根节点的子树符合要求所需的最少节点，对于当前节点所有子节点的 f 值排序，选前 k 个人的答案相加即可，$k =$ 子节点个数 *T% 向上取整。*

【参考代码】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
vector<int>a[maxn];
int n,t,x;
int dfs(int u,int fa){
    vector<int>tmp;
    int m=a[u].size();
```

```
    if (m==0) return 1;
    int tot=0;
    for (int i=0;i<=m-1;i++){
        int v=a[u][i];
        if (v!=fa){
            tmp.push_back(dfs(v,u));
        }
    }
    sort(tmp.begin(),tmp.end());
    int k=(t*m-1)/100+1;
    for (int i=0;i<=k-1;i++)
        tot+=tmp[i];
    return tot;
}
int main(){
    while (scanf("%d%d",&n,&t)!=EOF){
        if (n==0 && t==0) break;
        for (int i=0;i<=n;i++) a[i].clear();
        for (int i=1;i<=n;i++){
            scanf("%d",&x);
            a[x].push_back(i);
        }
        printf("%d\n",dfs(0,-1));
    }
}
```

【参考代码二】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
vector<int>a[maxn];
int n,T,x;
int dfs(int u,int fa){
    int m=a[u].size();
    if (m==0) return 1;
    vector<int>tmp;
    for (int i=0;i<m;i++){
        int v=a[u][i];
        if (v!=fa)
            tmp.push_back(dfs(v,u));
    }
    sort(tmp.begin(),tmp.end());
    int k=(m*T-1)/100+1;
    int tot=0;
    for (int i=0;i<k;i++){
        tot+=tmp[i];
    }
    return tot;
}
int main(){
    while (scanf("%d%d",&n,&T)!=EOF){
        if (n==0&&T==0) break;
        for (int i=0;i<=n;i++) a[i].clear();
```

```
        for (int i=1;i<=n;i++) {
            scanf("%d",&x);
            a[x].push_back(i);
        }
        printf("%d\n",dfs(0,-1));
    }
}
```

例题 POJ1848

【题目描述】

给你一棵 N 个节点的树，求最少添加几条边可以让每个点都在且仅在一个环中

【解题思路】

```
f[i][0]表示i为根的子树，保证每个点在且仅在一个环中要加的最少边数
f[i][1]表示i为根的子树，除了根节点外的点在且仅在一个环中要加的最少边数
f[i][2]表示i为根的子树，除了根所在的一条链外(大于等于两个点)的点在且仅在一个环中要加的最少边数
状态转移有四种：
f[i][1]+=f[son][0]
f[i][2]={f[son][0]}+min(f[son1][1],f[son1][2])
f[i][0]=min({f[son][0]}+f[son1][2])+1
f[i][0]=min(f[son][0]+min(f[son1][1],f[son1][2])+min(f[son2][1],f[son2][2]))+1
初值：
f[i][0]=inf;
f[i][1]=0;
```

[图解]

[POJ 1848 （一道不错的树形dp） - AC_Von - 博客园 (cnblogs.com)]

【代码】

```
#include<cstdio>
#include<vector>
#include<iostream>
using namespace std;
const int inf=10000;
int head[200],nxt[200],f[200][3],num;
bool vis[200];
struct node{
    int nxt,to;
}edge[200];
void addv(int x,int y){
    edge[++num].to=y;
    edge[num].nxt=head[x];
    head[x]=num;
}
void dfs(int x){
    int sum=0;
    vector <int> a;
    for (int i=head[x];i;i=edge[i].nxt){
```

```
        int v=edge[i].to;
        if (!vis[v]){
            vis[v]=1;
            dfs(v);
            sum+=f[v][0];
            a.push_back(v);
        }
    }
    f[x][1]=f[x][0]=f[x][2]=inf;
    f[x][1]=min(f[x][1],sum);
    int n=a.size();
    if (n==0) {
        f[x][1]=0;
        return ;
    }
    for (int i=0;i<n;i++){
        int j=a[i];
        f[x][2]=min(f[x][2],sum-f[j][0]+min(f[j][1],f[j][2]));
        f[x][0]=min(f[x][0],sum-f[j][0]+f[j][2]+1);
    }
    for (int i=0;i<n;i++)
        for (int j=0;j<n;j++){
            if (i==j) continue;
            int k1=a[i];
            int k2=a[j];
            f[x][0]=min(f[x][0],sum-f[k1][0]-f[k2][0]+min(f[k1][1],f[k1]
[2])+min(f[k2][1],f[k2][2])+1);
        }
}
int main(){
    int n;
    scanf("%d",&n);
    int num=0;
    int x,y;
    for (int i=1;i<n;i++){
        scanf("%d%d",&x,&y);
        addv(x,y);
        addv(y,x);
    }
    vis[1]=1;
    dfs(1);
    if (f[1][0]>=inf) printf("-1\n");else printf("%d\n",f[1][0]);
}
//题意：给定一棵n个节点，n-1条边的树 ，求添加最少的边使得每个节点正好只在一个环中
//题解：f[i][0],f[i][1],f[i][2]分别表示i节点的子树都符合，除了i节点符合，除了i节点所在的某
一条链都符合

/*
8
8 2
1 2
2 4
3 4
4 6
5 6
5 7
```

```
2
*/
```

【代码二】

```
#include<cstdio>
#include<iostream>
#include<vector>
using namespace std;
const int inf=1e9;
vector<int> a[200];
int m,x,y,f[200][3];
void dfs(int u,int fa){
    f[u][0]=f[u][1]=f[u][2]=inf;
    int n=a[u].size();
    if (n==0){
        f[u][1]=0;
        return;
    }
    int tot=0;
    for (int j=0;j<=n-1;j++){
        int v=a[u][j];
        if (v==fa) continue;
        dfs(v,u);
        tot+=f[v][0];
    }
    f[u][1]=tot;
    for (int j=0;j<=n-1;j++){
        int v=a[u][j];
        if (v==fa) continue;
        f[u][2]=min(f[u][2],tot-f[v][0]+min(f[v][1],f[v][2]));
        f[u][0]=min(f[u][0],tot-f[v][0]+f[v][2]+1);
    }
    for (int j1=0;j1<=n-1;j1++)
        for (int j2=0;j2<=n-1;j2++){
            int v1=a[u][j1];
            int v2=a[u][j2];
            if (v1==fa || v2==fa ||v1==v2) continue;
            f[u][0]=min(f[u][0],tot-f[v1][0]-f[v2][0]+min(f[v1][1],f[v1]
[2])+min(f[v2][1],f[v2][2])+1);
        }
}

int main(){
    scanf("%d",&m);
    for (int i=1;i<=m-1;i++){
        scanf("%d%d",&x,&y);
        a[x].push_back(y);
        a[y].push_back(x);
    }
    dfs(1,0);
    if (f[1][0]<inf) printf("%d\n",f[1][0]); else printf("-1\n");
}
```

【参考代码三】

```
#include<cstdio>
#include<vector>
using namespace std;
const int inf=1e5;
int n,x,y,f[200][3];
vector<int>a[200];
void dfs(int u,int fa){

    int m=a[u].size();

    int tot=0;
    for (int i=0;i<m;i++){
        int v=a[u][i];
        if (v!=fa){
            dfs(v,u);
            tot+=f[v][0];
        }
    }
    f[u][0]=f[u][1]=f[u][2]=inf;
    f[u][1]=min(f[u][1],tot);
    if (m==0) {
        f[u][1]=0;
        return;
    }
    for (int i=0;i<m;i++){
        int v=a[u][i];
        if (v!=fa){
            f[u][0]=min(f[u][0],tot-f[v][0]+f[v][2]+1);
            f[u][2]=min(f[u][2],tot-f[v][0]+min(f[v][1],f[v][2]));
        }
    }

    for (int i=0;i<m;i++)
        for (int j=0;j<m;j++){
            if (i==j) continue;
            int v1=a[u][i];
            int v2=a[u][j];
            if (v1!=fa&&v2!=fa){
                f[u][0]=min(f[u][0],tot-f[v1][0]-f[v2][0]+min(f[v1][1],f[v1]
[2])+min(f[v2][1],f[v2][2])+1);
            }

        }
}
int main(){
    scanf("%d",&n);
    for (int i=1;i<=n-1;i++){
        scanf("%d%d",&x,&y);
        a[x].push_back(y);
        a[y].push_back(x);
    }
    dfs(1,0);
    if (f[1][0]<inf) printf("%d\n",f[1][0]);else printf("-1\n");
```

```
}
```

AT_dp_v Subtree

【题目翻译】

[Subtree - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)](https://www.luogu.com.cn/problem/AT_dp_v)

【解题思路】

树型DP+换根操作

状态描述：

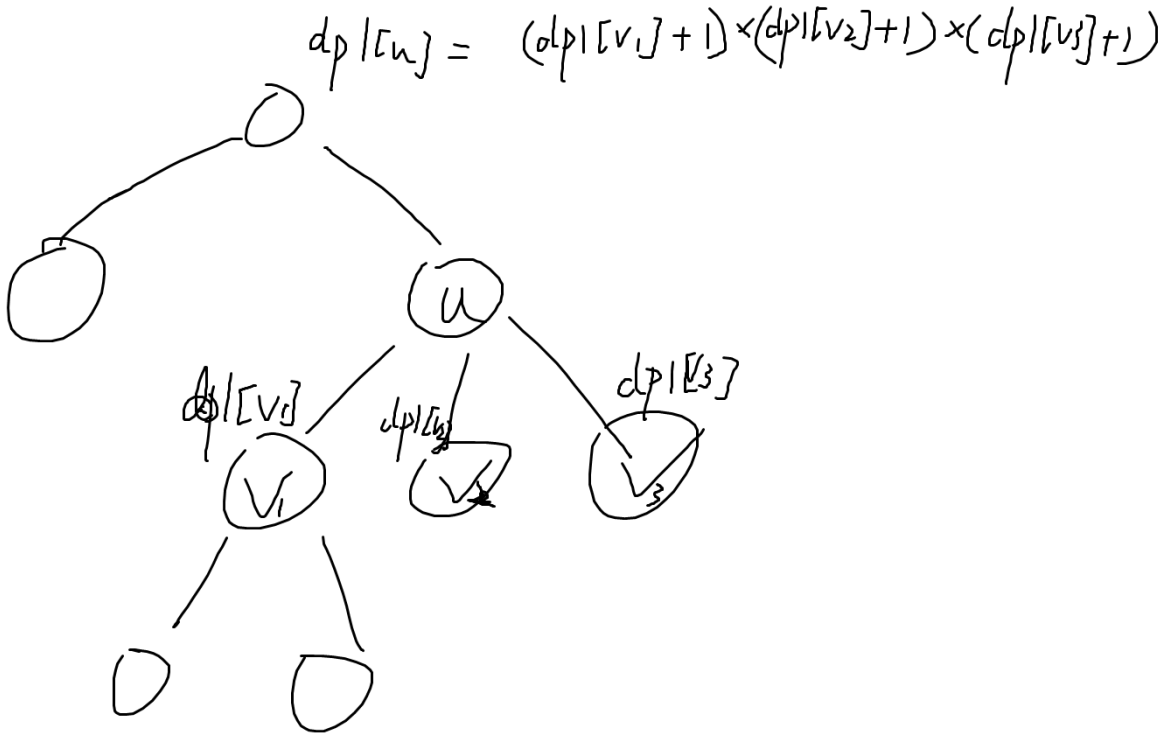
$dp1[i]$ 表示以 i 为根结点的子树的染色方案数（i 必须染成黑色）

$dp2[i]$ 表示以 i 为根结点的子树外的结点和 i 的染色方案数（i 必须染成黑色）

状态转移：

使用 dfs 来实现，对于两个状态，跑两次 $dfs1$ 、$dfs2$

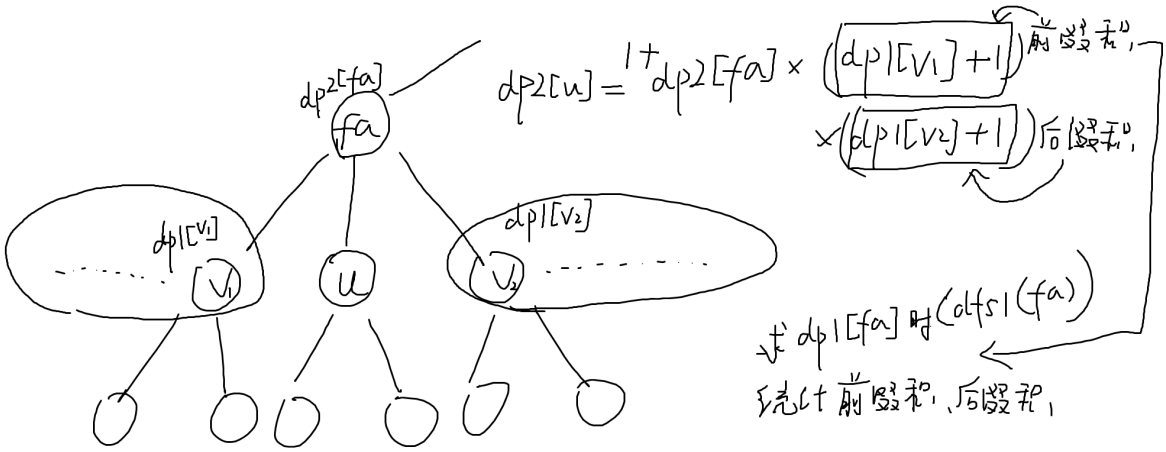
首先来看 $dp1[i]$ 的求取，如下图为例：（$dfs1$ 实现）



概括起来状态方程：$dp1[u] = \prod(dp1[v]+1)$（v是 u的 儿 子）

根据乘法原理，如上图每个子树的方案彼此独立，然后 $dp1[u]$ 强制要求 u 染成黑色，因此要+1，表示不染色的情况。

再来看 $dp2[i]$ 的求取，如下图为例：（$dfs2$ 实现）



概括起来转移方程就是：$dp2[u] = dp2[fa] \times \prod(dp1[v]+1)+1$

其中 fa 是 u 的父亲，v 是 u 的兄弟。表示对于每个子树外的节点，可以分为**兄弟子树中的节点**和**父亲以上的节点**两类，用父亲以上的节点的 dp2 值乘上**兄弟节点子树的可能数 +1 的积**。最后总的答案 +1 表示 u 子树外结点都不染色的可能。

终值 就是 $dp1[u] \times dp2[u]$

时间复杂度、**空间复杂度** 都是 $O(n)$

【参考程序】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+10;
vector<long long >a[maxn];
vector<long long>mul1[maxn],mul2[maxn];
long long dp1[maxn],dp2[maxn];
long long M,n,x,y;
void dfs1(int u,int fa){
    //printf("%d %d\n",u,fa);
    mul1[u].resize(a[u].size(),1);
    mul2[u].resize(a[u].size(),1);
    if (a[u].size()==1&&a[u][0]==fa){
        dp1[u]=1;
        return ;
    }
    dp1[u]=1ll;
    for (int i=0;i<a[u].size();i++){
        if (a[u][i]!=fa){
            dfs1(a[u][i],u);
            dp1[u]=dp1[u]*(dp1[a[u][i]]+1)%M;

        }
    }
    int nn=a[u].size();
    for (int i=1;i<nn;i++){
        mul1[u][i]=mul1[u][i-1];
        if (a[u][i-1]!=fa)
            mul1[u][i]=mul1[u][i]*(dp1[a[u][i-1]]+1)%M;
    }
    for (int i=nn-2;i>=0;i--){
        mul2[u][i]=mul2[u][i+1];
        if (a[u][i+1]!=fa)
            mul2[u][i]=mul2[u][i]*(dp1[a[u][i+1]]+1)%M;
```

```
    }
}
void dfs2(int u,int fa,int id){
    if (fa!=-1) {
            dp2[u]=(dp2[fa]*mul1[fa][id]%M*mul2[fa][id]%M+1)%M;
    }
    else {
        dp2[u]=1;
    }
    int nn=a[u].size();
    for (int i=0;i<nn;i++){
        if (a[u][i]!=fa)
            dfs2(a[u][i],u,i);
    }
}
int main(){
    scanf("%lld%lld",&n,&M);
    for (int i=1;i<=n-1;i++){
        scanf("%lld%lld",&x,&y);
        a[x-1].push_back(y-1);
        a[y-1].push_back(x-1);
    }
    dfs1(0,-1);
    dfs2(0,-1,-1);
    long long ans=0;
    for (int i=0;i<n;i++)
        printf("%lld\n",(dp1[i]*dp2[i])%M);

}
```

【参考代码二】

```
#include<bits/stdc++.h>
using namespace std;
const int maxn=1e5+50;
long long dp1[maxn],dp2[maxn],n,M,x,y;
vector<long long>mul1[maxn],mul2[maxn],a[maxn];
void dfs1(int u,int fa){

    int m=a[u].size();
    if (m==1&&a[u][0]==fa){
        dp1[u]=1;
        return;
    }
    dp1[u]=1;
    for (int i=0;i<m;i++){
        int v=a[u][i];
        if (v!=fa) {
            dfs1(v,u);
            dp1[u]=dp1[u]*(1+dp1[v])%M;
        }
    }
    mul1[u].resize(a[u].size(),1);
    mul2[u].resize(a[u].size(),1);
    for (int i=1;i<m;i++){
        mul1[u][i]=mul1[u][i-1];
```

```
        int v=a[u][i-1];
        if (v!=fa)
            mul1[u][i]=mul1[u][i]*(dp1[v]+1)%M;
    }
    for (int i=m-2;i>=0;i--){
        mul2[u][i]=mul2[u][i+1];
        int v=a[u][i+1];
        if (v!=fa)
            mul2[u][i]=mul2[u][i]*(dp1[v]+1)%M;
    }

}

void dfs2(int u,int fa,int id){
    if (fa==-1){
        dp2[u]=1;
    }
    else{
        dp2[u]=(1+dp2[fa]*mul1[fa][id]%M*mul2[fa][id]%M)%M;
    }
    int m=a[u].size();
    for (int i=0;i<m;i++){
        int v=a[u][i];
        if (v!=fa)
            dfs2(v,u,i);
    }
}

int main(){
    scanf("%lld%lld",&n,&M);
    for (int i=1;i<=n-1;i++){
        scanf("%lld%lld",&x,&y);
        a[x].push_back(y);
        a[y].push_back(x);
    }
    dfs1(1,-1);
    dfs2(1,-1,-1);
    for (int i=1;i<=n;i++){
        printf("%lld\n",dp1[i]*dp2[i]%M);
    }
}
```

例题 POJ1770

【题目描述】

在 n 个数 $a[i]$ 中挑选和尽可能大的数，并且这些任意两个差值要等于 $b[j]$

【解题思路】

$f[i][0..1]$ 分别表示以 i 为根节点的子树中（根节点选不选）所能获得的最大和

```
f[i][0]=max{f[son][1],f[son][0]};f[i][1]=max{f[son][0]}+a[i];
```

访问son节点这里用了邻接表

```
状态：
f[i][0]表示i为根的子树，i不选能够获得的最大和
f[i][1]表示i为根的子树，i选能够获得的最大和
```

转移：

$f[i][0] = \sum_{son=1}^{k} max(f[son][1], f[son][0])$

$f[i][1] = \sum_{son=1}^{k} f[son][0]$

初值：

$f[i][1] = a[i]$

【代码】

```
#include<cstdio>
#include<algorithm>
#include<iostream>
#include<cstring>
using namespace std;
struct node{
    int to,nxt;
}edge[41000];
int num,n,m,vis[410],a[210],b[210],f[400][3],head[210];
void add_e(int x,int y){
    num++;
    edge[num].to=y;
    edge[num].nxt=head[x];
    head[x]=num;
}
void dfs(int fa,int x,int y){
    if (f[x][y]>=0) return;
    int v;
    int ans=0;
    for (int i=head[x];i;i=edge[i].nxt){
        v=edge[i].to;
        if (fa!=v){
            dfs(x,v,0);
            dfs(x,v,1);
            if (y==1) ans+=f[v][0];else ans+=max(f[v][0],f[v][1]);
        }
    }
    if (y==1)f[x][1]=ans+a[x]; else f[x][0]=ans;
}
int main(){
    while (~scanf("%d%d",&n,&m)){
        if (n==0 && m==0) break;
        num=0;
        for (int i=1;i<=n;i++)f[i][0]=f[i][1]=-1;
        memset(head,0,sizeof(head));
        for (int i=1;i<=n;i++){
            scanf("%d",&a[i]);
        }
        for (int i=1;i<=m;i++)  scanf("%d",&b[i]);
        for (int i=1;i<=n-1;i++)
            for (int j=i+1;j<=n;j++)
```

```
                for (int k=1;k<=m;k++){
                    if (abs(a[i]-a[j])==b[k]){
                        add_e(i,j);
                        add_e(j,i);
                    }
            }
    dfs(0,1,0);dfs(0,1,1);
    printf("%d\n",max(f[1][0],f[1][1]));
    }
}
```

注意：树上搜索一定会使用fa这个参数，如果不用，比如这题用布尔型数组标记i节点有没有访问过，结果dfs(1,0),dfs(1,1)只访问了一次，就导致了错误

UVA1220

独立集：是指图 G 中两两互不相邻的顶点构成的集合。

题意：找出一个树的最大点独立集，并输出这个最大点独立集是否唯一，类似[P1352 没有上司的舞会](),但多了唯一性的判断

【解题思路】

[UVA1220 Hali-Bula的晚会 Party at Hali-Bula - 洛谷 | 计算机科学教育新生态 (luogu.com.cn)]()

例题 POJ3342（和上面同一题）

【题目描述】

n个公司员工参加舞会，员工和直属老板不能同时参加，问能参加的最多人数，并且回答方案是否唯一

【解题思路】

状态：

$f[i][0]$表示i为根的子树，i不选能取得的最大节点数

$f[i][1]$表示i为根的子树，i选能取得的最大节点数

$p[i][0/1]$ 表示方案是否唯一

转移：

```
f[i][0]=sum(max(f[son][1],f[son][0]))
f[i][1]=sum(f[son][0])
存在不唯一：
f[i][0]==f[i][1]
pd[son][0]==1
pd[son][1]==1
```

【代码】

```
#include<cstdio>
#include<map>
#include<vector>
#include<iostream>
```

```
#include<cstring>
using namespace std;
map<string,int>mp;
char s[110],s1[110],s2[110];
int f[300][2],n,num;
int  pd[300][3];
vector<int>a[300];

void dfs(int u){
    int n=a[u].size();
    for (int i=0;i<n;i++)
    {
         int v=a[u][i];
         dfs(v);
         f[u][0]+=max(f[v][1],f[v][0]);
         if (f[v][1]==f[v][0]) pd[u][0]=0;
         else {
            if (f[v][1]<f[v][0]){
                if(pd[v][0]==0)pd[u][0]=0;
             }
            else {
                if (pd[v][1]==0) pd[u][0]=0;
             }
         }
         f[u][1]+=f[v][0];
         if (pd[v][0]==0) pd[u][1]=0;
    }
}

int main(){
    while (~scanf("%d",&n)){
        bool flag=1;
        if(n==0)break;
        for (int i=0;i<=n;i++){
            a[i].clear();
            f[i][0]=0;
            f[i][1]=1;
            pd[i][0]=1;
            pd[i][1]=1;
        }
        num=0;
        mp.clear();
        scanf("%s",s);
        mp[s]=++num;
        for (int i=1;i<=n-1;i++){
            scanf("%s%s",s1,s2);
            if (mp[s1]==0) mp[s1]=++num;
            if (mp[s2]==0) mp[s2]=++num;
            a[mp[s2]].push_back(mp[s1]);
        }
        dfs(1);

        if (f[1][0]==f[1][1]) printf("%d No\n",f[1][0]);
        else if (f[1][0]>f[1][1]) {
            if (pd[1][0]==0) printf("%d No\n",f[1][0]);
            else printf("%d Yes\n",f[1][0]);
        }
        else if (f[1][0]<f[1][1])
```

```
        {
            if (pd[1][1]==0)printf("%d No\n",f[1][1]);
            else printf("%d Yes\n",f[1][1]);
        }

    }
}
// 题意：  公司职员参加舞会，求能挑选的最多人数，员工和直属老板不能同时存在，并判断方案是否唯一
// 解题思路：f[i][0..1]表示i为根节点的子树（根节点选或不选）能选取的最多数量；pd[i][0..1]表示最优方案是否唯一
```

【代码2】

```
#include<cstdio>
#include<map>
#include<vector>
#include<iostream>
#include<cstring>
using namespace std;
map<string,int>mp;
char s[110],s1[110],s2[110];
int f[300][2],n,num;
vector<int>a[300];
void dfs(int u){
    int n=a[u].size();

    for (int i=0;i<n;i++)
    {
         int v=a[u][i];
         dfs(v);
         f[u][0]+=max(f[v][1],f[v][0]);
         f[u][1]+=f[v][0];
    }

}
bool check(){
    for(int i=1;i<=n;i++)
    if(f[i][0]>=f[i][1])
       for(int j=0;j<a[i].size();j++)
       {
            int v=a[i][j];
            if(f[v][1]==f[v][0])
                return 0;
       }
    return 1;
}
int main(){
    while (~scanf("%d",&n)){
        bool flag=1;
        if(n==0)break;
        for (int i=0;i<=n;i++){
            a[i].clear();
            f[i][0]=0;
            f[i][1]=1;
        }
        num=0;
```

```
        mp.clear();
        bool no_flag=0;
        scanf("%s",s);
        mp[s]=++num;
        for (int i=1;i<=n-1;i++){
            scanf("%s%s",s1,s2);
            if (mp[s2]==0) mp[s2]=++num;
            if (mp[s1]==0) mp[s1]=++num;
            a[mp[s2]].push_back(mp[s1]);
        }
        dfs(1);
        if (f[1][0]==f[1][1])flag=0;
        if (flag) flag=check();
        if (!flag) printf("%d No\n",max(f[1][0],f[1][1]));else printf("%d
Yes\n",max(f[1][1],f[1][0]));
    }
}
// 题意： 公司职员参加舞会，求能挑选的最多人数，员工和直属老板不能同时存在，并判断方案是否唯一
// 解题思路：f[i][0..1]表示i为根节点的子树（根节点选或不选）能选取的最多数量；对于每个节点i，
如果f[i][0]>=f[i][1]才有可能存在多种方案，然后判断f[son][0]==f[son][1]
```

例题 POJ2486

【题目描述】

给你一棵 n 个节点的带权树，n 条边，从 1 号点出发走 k 步能够取得的最大权值和

【解题思路】

```
f[u][j][0..1]表示u为根的子树走了j步回到过u的最大权值和
(1) f[u][j+2][0]=max(f[u][j+2][0],f[u][j-k][0]+f[son][k][0]
(2) f[u][j+2][1]=max(f[u][j+2][1],f[u][j-k][1]+f[son][k][0]
(3) f[u][j+1][1]=max(f[u][j+1][1],f[u][j-k][0]+f[son][k][1]
(1)表示u出发回到u(j-k)步，u到v来回2步，v出发回到v的k步
(2)表示u出发到v的1步，v出发回到v的k步，在从v到u的1步，u出发不回来的j-k步
(3)表示u出发回到u的j-k步，再到v的1步，v出发不回来的k步
```

【代码】

```
#include<cstdio>
#include<cstring>
#include<iostream>
using namespace std;
int num,n,m,a[200],f[200][200][2],head[200];
struct node {
    int to,nxt;
}b[200];
void add_edge(int u,int v){
    num++;
    b[num].to=v;
    b[num].nxt=head[u];
```

```
    head[u]=num;
}
void dfs(int fa,int u){
    for (int i=0;i<=m;i++) f[u][i][0]=f[u][i][1]=a[u];
    for (int i=head[u];i;i=b[i].nxt){
        int v=b[i].to;
        if (fa==v)continue;
        dfs(u,v);
        for (int j=m;j>=0;j--)
            for (int k=0;k<=j;k++){
                    f[u][j+2][0]=max(f[u][j+2][0],f[u][j-k][0]+f[v][k][0]);
                    f[u][j+1][1]=max(f[u][j+1][1],f[u][j-k][0]+f[v][k][1]);
                    f[u][j+2][1]=max(f[u][j+2][1],f[u][j-k][1]+f[v][k][0]);
            }
    }
}
int main(){
    while (~scanf("%d%d",&n,&m)){
        memset(head,0,sizeof(head));
        num=0;
        int u,v;
        for (int i=1;i<=n;i++)scanf("%d",&a[i]);
        for (int i=1;i<=n-1;i++){
            scanf("%d%d",&u,&v);
            add_edge(u,v);
            add_edge(v,u);
        }
        dfs(0,1);
        printf("%d\n",max(f[1][m][0],f[1][m][1]));
    }
}
```

例题 POJ3140

【题目描述】

n个节点，n-1条边，求去除一条边后两颗子树权值和的最小差值

【解题思路】

树形DP，回溯的时候统计u为根的子树权值和，统计u和v边断掉，两颗子树的差值更新最优值

注意：多组数据，不一定要重置所有的数组值，本题只需重置head数组

【代码】

```
#include<cstdio>
#include<cmath>
#include<iostream>
#include<cstring>
using namespace std;
const int N=2000000;
const int M=2000000;
```

```
long long sum,f[N],ans;
int head[N],a[N],num;
struct node{
    int to,nxt;
}b[M];
long long Abs(long long x){
    return x>0?x:-x;
}
void add_edge(int u,int v){
    num++;
    b[num].to=v;
    b[num].nxt=head[u];
    head[u]=num;
}
void dfs(int fa,int u){
    //f[u]=a[u];
    for (int i=head[u];i;i=b[i].nxt){
        int v=b[i].to;
        if (fa==v) continue;
        dfs(u,v);
        f[u]+=f[v];

    }
    ans=min(ans,Abs(sum-f[u]-f[u]));
}
int main(){
    int T=0,n,m,u,v;
    while (~scanf("%d%d",&n,&m)){

        memset(head,0,sizeof(head));

        if (n==0&&m==0)break;
        sum=0;
        num=0;
        T++;
        for (int i=1;i<=n;i++){
            scanf("%d",&a[i]);
            f[i]=a[i];
            sum+=a[i];
        }
        for (int i=1;i<=m;i++){
            scanf("%d%d",&u,&v);
            add_edge(u,v);
            add_edge(v,u);
        }
        ans=sum;
        dfs(0,1);

        printf("Case %d: %lld\n",T,ans);
    }
}
```

完成vjudge题目列表

[DP_Round #2 - Virtual Judge (vjudge.net)]

18.4 小结

|树形DP的定义：树上进行的、父节点依赖子节点的DP
|时间复杂度：和树上节点数量以及父子节点间决策数量有关
|用途：很多问题都可以转成搜索树，也就是树形DP的范畴，所以应用场合很多
树形DP其实和之前的DP原理上还是一样，求解的顺序还是遵循依赖关系，由叶节点求到树根，因此它可以融合其他DP问题，把线性上进行的转移放到树上即可